

Travaux Pratiques

Module M214 - Application Cloud Native

Séance 3 : Middlewares & Authentification JWT

PARTIE 1 : Atelier Guidé - API avec JWT

Créez une API sécurisée avec authentification JWT.

Étape 1 : Initialiser le projet

```
1 mkdir api-jwt
2 cd api-jwt
3 npm init -y
4 npm install express mongoose jsonwebtoken bcryptjs cors
  morgan
```

Étape 2 : Créer le modèle User

Créez models/User.js :

```
1 const mongoose = require('mongoose');
2
3 const userSchema = new mongoose.Schema({
4   nom: { type: String, required: true },
5   email: { type: String, required: true, unique: true,
6     lowercase: true },
7   password: { type: String, required: true, minlength: 6 },
8   role: { type: String, enum: ['user', 'admin'], default: '
9     user' }
10 }, { timestamps: true });
11
12 module.exports = mongoose.model('User', userSchema);
```

✎ Étape 3 : Configuration de base - index.js

```
1 const express = require('express');
2 const mongoose = require('mongoose');
3 const cors = require('cors');
4 const morgan = require('morgan');
5
6 const app = express();
7
8 // Middlewares globaux
9 app.use(cors());
10 app.use(morgan('dev'));
11 app.use(express.json());
12
13 // Connexion MongoDB
14 mongoose.connect('mongodb://localhost:27017/api-jwt')
15   .then(() => console.log('MongoDB connecte'));
16
17 // Variable pour JWT (en production: process.env.JWT_SECRET)
18 const JWT_SECRET = 'ma_cle_secrete_super_longue_123!';
19
20 app.listen(3000, () => console.log('Serveur sur port 3000'));
```

 Étape 4 : Route d'inscription

```
1 const bcrypt = require('bcryptjs');
2 const jwt = require('jsonwebtoken');
3 const User = require('../models/User');
4
5 app.post('/api/auth/register', async (req, res) => {
6   try {
7     const { nom, email, password } = req.body;
8
9     // Verifier si l'email existe
10    const existingUser = await User.findOne({ email });
11    if (existingUser) {
12      return res.status(400).json({ error: 'Email deja
utilise' });
13    }
14
15    // Hasher le mot de passe
16    const hashedPassword = await bcrypt.hash(password,
10);
17
18    // Creer l'utilisateur
19    const user = await User.create({
20      nom, email, password: hashedPassword
21    });
22
23    // Generer le token
24    const token = jwt.sign(
25      { id: user._id, email: user.email, role: user.
role },
26      JWT_SECRET,
27      { expiresIn: '24h' }
28    );
29
30    res.status(201).json({ success: true, token });
31  } catch (error) {
32    res.status(500).json({ error: error.message });
33  }
34 });
```

➡ Étape 5 : Route de connexion

```
1 app.post('/api/auth/login', async (req, res) => {
2   try {
3     const { email, password } = req.body;
4
5     // Trouver l'utilisateur
6     const user = await User.findOne({ email });
7     if (!user) {
8       return res.status(401).json({ error: 'Email ou
mot de passe incorrect' });
9     }
10
11    // Verifier le mot de passe
12    const isMatch = await bcrypt.compare(password, user.
password);
13    if (!isMatch) {
14      return res.status(401).json({ error: 'Email ou
mot de passe incorrect' });
15    }
16
17    // Generer le token
18    const token = jwt.sign(
19      { id: user._id, email: user.email, role: user.
role },
20      JWT_SECRET,
21      { expiresIn: '24h' }
22    );
23
24    res.json({ success: true, token, user: { nom: user.
nom, email: user.email } });
25  } catch (error) {
26    res.status(500).json({ error: error.message });
27  }
28 });
```

Étape 6 : Middleware d'authentification

Créez middlewares/auth.js :

```
1 const jwt = require('jsonwebtoken');
2 const JWT_SECRET = 'ma_cle_secrete_super_longue_123!';
3
4 const authenticateToken = (req, res, next) => {
5   const authHeader = req.headers['authorization'];
6
7   if (!authHeader) {
8     return res.status(401).json({ error: 'Token manquant' });
9   }
10
11   const token = authHeader.split(' ')[1]; // "Bearer <
  token>"
12
13   try {
14     const decoded = jwt.verify(token, JWT_SECRET);
15     req.user = decoded;
16     next();
17   } catch (error) {
18     res.status(401).json({ error: 'Token invalide ou
  expire' });
19   }
20 };
21
22 module.exports = authenticateToken;
```

Étape 7 : Routes protégées

```
1 const authenticateToken = require('./middlewares/auth');
2
3 // Route protegee - Profil utilisateur
4 app.get('/api/profile', authenticateToken, async (req, res)
5   => {
6     const user = await User.findById(req.user.id).select('-
7     password');
8     res.json({ user });
9   });
10
11 // Middleware d'autorisation par role
12 const authorizeRole = (...roles) => {
13   return (req, res, next) => {
14     if (!roles.includes(req.user.role)) {
15       return res.status(403).json({ error: 'Acces
16       interdit' });
17     }
18     next();
19   };
20 };
21
22 // Route admin seulement
23 app.get('/api/admin/users', authenticateToken, authorizeRole
24   ('admin'), async (req, res) => {
25     const users = await User.find().select('-password');
26     res.json({ users });
27   });
```

Étape 8 : Gestion des erreurs

```
1 // Middleware 404
2 app.use((req, res) => {
3   res.status(404).json({ error: 'Route non trouvee' });
4 });
5
6 // Middleware d'erreur global
7 app.use((err, req, res, next) => {
8   console.error(err.stack);
9   res.status(500).json({ error: 'Erreur serveur' });
10 });
```

✍ Étape 9 : Tester avec Postman

1. Register : POST `http://localhost:3000/api/auth/register`

```
1 { "nom": "Ahmed", "email": "ahmed@mail.com", "password":  
  "123456" }
```

2. Login : POST `http://localhost:3000/api/auth/login`

```
1 { "email": "ahmed@mail.com", "password": "123456" }
```

3. Profile : GET `http://localhost:3000/api/profile`

Header : Authorization: Bearer <token_recu>

PARTIE 2 : Exercice Autonome

Middlewares Personnalisés

- Q1.** Créez un middleware `logger` qui affiche pour chaque requête : la date/heure, la méthode HTTP, l'URL, et l'IP du client.
- Q2.** Créez un middleware `validateRegister` qui vérifie que `req.body` contient : nom (min 2 caractères), email (contient @), password (min 6 caractères). Retournez les erreurs en JSON avec status 400.
- Q3.** Que se passe-t-il si on oublie d'appeler `next()` dans un middleware ?
- Q4.** Quelle est la différence entre `app.use(middleware)` et `app.get('/route', middleware, handler)` ?
- Q5.** Pourquoi doit-on déclarer `express.json()` avant les routes POST ?
- Q6.** Écrivez un middleware `requestId` qui génère un ID unique pour chaque requête et l'ajoute à `req.id`.

JWT & Authentification

- Q7.** Quelles sont les 3 parties d'un token JWT ? Décrivez brièvement chacune.
- Q8.** Pourquoi ne doit-on jamais stocker de données sensibles (mot de passe, carte bancaire) dans le payload JWT ?
- Q9.** Écrivez la fonction `jwt.sign()` pour créer un token avec : id utilisateur, email, rôle, expiration de 7 jours.
- Q10.** Dans le middleware d'authentification, comment extrait-on le token du header `Authorization: Bearer eyJhbG...` ?
- Q11.** Quelle est la différence entre les codes HTTP 401 (Unauthorized) et 403 (Forbidden) ?
- Q12.** Pourquoi utilise-t-on `bcrypt.hash()` pour stocker les mots de passe au lieu de les stocker en clair ?
- Q13.** Complétez : Pour vérifier un token, on utilise `jwt._____ (token, secret)`.
- Q14.** Écrivez un middleware `authorizeRole` qui accepte une liste de rôles autorisés et bloque (403) si le rôle de l'utilisateur n'est pas dans la liste.

PARTIE 3 : Challenge

🔥 API Blog Sécurisée

Créez une API de blog avec authentification JWT complète.

Modèles :

- **User** : nom, email (unique), password, role (user/admin), createdAt
- **Article** : titre, contenu, auteur (ref User), categorie, published (boolean), createdAt

- Q1.** Créez les schémas Mongoose pour User et Article (avec la référence auteur).
- Q2.** Implémentez les routes `POST /api/auth/register` et `POST /api/auth/login` avec hashage bcrypt et génération JWT.
- Q3.** Créez le middleware `authenticateToken` qui vérifie le JWT et ajoute `req.user`.
- Q4.** Implémentez `POST /api/articles` (protégée) qui crée un article avec l'auteur = utilisateur connecté (`req.user.id`).
- Q5.** Implémentez `GET /api/articles` (publique) qui retourne tous les articles publiés avec populate sur l'auteur (nom seulement).
- Q6.** Implémentez `PUT /api/articles/:id` (protégée) qui permet de modifier un article uniquement si l'utilisateur est l'auteur OU admin.
- Q7.** Implémentez `DELETE /api/articles/:id` (protégée, admin seulement).
- Q8.** Créez un middleware `isOwnerOrAdmin` qui vérifie si l'utilisateur est l'auteur de l'article ou un admin avant d'autoriser la modification.
- Q9.** Ajoutez une route `GET /api/articles/mes-articles` (protégée) qui retourne uniquement les articles de l'utilisateur connecté.
- Q10.** Implémentez un système de refresh token : créez une route `POST /api/auth/refresh` qui génère un nouveau token si l'ancien est sur le point d'expirer.